

Задание 1

Дан фрагмент на языке программы C++. Нужно:

1. Построить КС-грамматику для аналогичных фрагментов.
2. Построить трехадресный код для исходного фрагмента.
3. Написать список семантических проверок, которые можно применить к исходному фрагменту.
4. Построить атрибутивную грамматику описывающую синтаксически управляемую трансляцию в трехадресный код.

Пример задания 1

```
int a = 5, b;  
scanf("%d", &b);  
if (b == a) {  
    printf("test");  
} else {  
    a = b / 3 + 7;  
}
```

КС-грамматика

```
grammar example1;  
  
program  
    : statement_list  
    ;  
  
statement_list  
    : statement+  
    ;  
  
statement  
    : declarete_var_statement  
    | call_function  
    | if_statement  
    | assign_statement  
    ;  
  
declarete_var_statement  
    : type declarete_var (',' declarete_var)* ';' ;  
  
type returns
```

```

    : 'int'
    ;

declarete_var
    : ID ('=' NUMBER)?
    ;

call_function
    : ID '(' (arg_list+=argument (',' arg_list+=argument)*)? ')' ';'
    ;

argument
    : expr
    ;

if_statement
    : 'if' '(' logic_expr ')' '{' s1=statement_list '}' ('else' '{'
s2=statement_list '}')?
    ;

logic_expr
    : i1=ID '==' i2=ID
    ;

assign_statement
    : ID '=' expr ';'
    ;

expr
    : ID
    | NUMBER
    | STRING
    | '&'ID
    | e1=expr '/' e2=expr
    | e1=expr '+' e2=expr
    ;

ID
    : [a-zA-Z][a-zA-Z0-9_]*
    ;

STRING
    : '''~["]*'''
    ;

NUMBER
    : [0-9]+
    ;

```

```
: [ \t\r\n]+ -> skip  
;
```

Трехадресный код

```
Consts table:  
( 'd1', '%d' )  
( 'd2', 'test' )  
  
Codes table:  
( 'ASSIGN', 'a', '5' )  
( 'ADDR', 't1', 'b' )  
( 'PARAM', 'd1' )  
( 'PARAM', 't1' )  
( 'CALL', 'scanf', 2 )  
( 'EQ', 't2', 'b', 'a' )  
( 'IF', 't2', 'l1' )  
( 'DIV', 't3', 'b', '3' )  
( 'ADD', 't4', 't3', '7' )  
( 'ASSIGN', 'a', 't4' )  
( 'GOTO', 'l2' )  
( 'LABEL', 'l1' )  
( 'PARAM', 'd2' )  
( 'CALL', 'printf', 1 )  
( 'LABEL', 'l2' )
```

Семантические проверки

- Объявлена ли переменная
- Повторное объявление переменной
- Совместимость типов в выражении и операторах

Атрибутивная грамматика

```
grammar example1;  
  
options {  
    superClass = BaseParser;  
    language = Python3;  
}  
  
program  
    : statement_list {  
self.codes = $statement_list.result_codes.copy()  
    }  
    ;
```

```

statement_list returns [result_codes] locals [codes]
@init {
$codes = []
}
    : statement+ {
$result_codes = $codes.copy()
    }
    ;

statement
    : declarete_var_statement
    | call_function
    | if_statement
    | assign_statement
    ;

declarete_var_statement
    : type declarete_var[$type.declare_type] (','
declarete_var[$type.declare_type])* ';'
    ;

type returns [declare_type]
    : 'int' {
$declare_type = "int"
    }
    ;

declarete_var [declare_type]
    : ID ('=' NUMBER)? {
if $ID.text in self.names:
    self.errors.append(f"Variable {$ID.text} is already declared")
    return
else:
    self.names[$ID.text] = { 'type': $declare_type, 'init_value':
$NUMBER.text }
    if $NUMBER.text:
        $statement_list::codes.append(("ASSIGN", $ID.text, $NUMBER.text))
    }
    ;

call_function
    : ID '(' (arg_list+=argument (',' arg_list+=argument)*)? ')' ';' {
for argument in $arg_list:
    $statement_list::codes.append(("PARAM", argument.result_var))
$statement_list::codes.append(("CALL", $ID.text, len($arg_list)))
}
    ;

```

```

argument returns [result_var]
    : expr {
$result_var = $expr.result_var
    }
    ;

if_statement
    : 'if' '(' logic_expr ')' '{' s1=statement_list '}' ('else' '{'
s2=statement_list '}')? {
label_then = self.next_temporal_label()
label_finish = self.next_temporal_label()
$statement_list::codes.append(("IF", $logic_expr.result_var, label_then))
if $s2.text:
    $statement_list::codes.extend($s2.result_codes)
$statement_list::codes.append(("GOTO", label_finish))
$statement_list::codes.append(("LABEL", label_then))
$statement_list::codes.extend($s1.result_codes)
$statement_list::codes.append(("LABEL", label_finish))
    }
    ;

logic_expr returns [result_var]
    : i1=ID '==' i2=ID {
$result_var = self.next_temporal_variable()
$statement_list::codes.append(("EQ", $result_var, $i1.text, $i2.text))
    }
    ;

assign_statement
    : ID '=' expr ';' {
if $ID.text not in self.names:
    self.errors.append(f"Variable {$ID.text} is not declared")
    return
else:
    if $expr.result_type != self.names[$ID.text].get('type'):
        self.errors.append(f"Type mismatch")
        return
$statement_list::codes.append(("ASSIGN", $ID.text, $expr.result_var))
    }
    ;

expr returns [result_var, result_type]
    : ID {
if $ID.text not in self.names:
    self.errors.append(f"Variable {$ID.text} is not declared")
    return
else:
    $result_var = $ID.text
    $result_type = self.names[$ID.text].get('type')
    }

```

```

    | NUMBER {
$result_var = $NUMBER.text
$result_type = "int"
    }
    | STRING {
$result_var = self.next_temporal_const()
$result_type = "string"
self.consts.append(($result_var, $STRING.text))
    }
    | '&'ID {
if $ID.text not in self.names:
    self.errors.append(f"Variable {$ID.text} not declared")
    return
else:
    $result_type = "addr"
    $result_var = self.next_temporal_variable()
    $statement_list::codes.append(("ADDR", $result_var, $ID.text))
    }
    | e1=expr '/' e2=expr {
if $e1.result_type != $e2.result_type:
    self.errors.append(f"Type mismatch {$e1.result_type} /
{$e2.result_type}")
    return
else:
    $result_type = $e1.result_type
    $result_var = self.next_temporal_variable()
    $statement_list::codes.append(("DIV", $result_var, $e1.result_var,
$result_var))
    }
    | e1=expr '+' e2=expr {
if $e1.result_type != $e2.result_type:
    self.errors.append(f"Type mismatch {$e1.result_type} +
{$e2.result_type}")
    return
else:
    $result_type = $e1.result_type
    $result_var = self.next_temporal_variable()
    $statement_list::codes.append(("ADD", $result_var, $e1.result_var,
$result_var))
    }
    ;

ID
: [a-zA-Z][a-zA-Z0-9_]*
;

STRING
: '"'~["]*'"
;

```

NUMBER

: [0-9]+

;

WS

: [\t\r\n]+ -> skip

;

Задание 2

Даны два понятия. Необходимо построить цепочку связей между понятиями, используя отношения Is-a, Has-a / Part-of, Instance-of. Для всех используемых понятий записать определение.

Распределение баллов за выполненные задания

- Задание 1
 - Пункт 1 - 1 бал
 - Пункт 2 - 2 бала
 - Пункт 3 - 1 бал
 - Пункт 4 - 5 баллов
- Задание 2 - 1 бал